

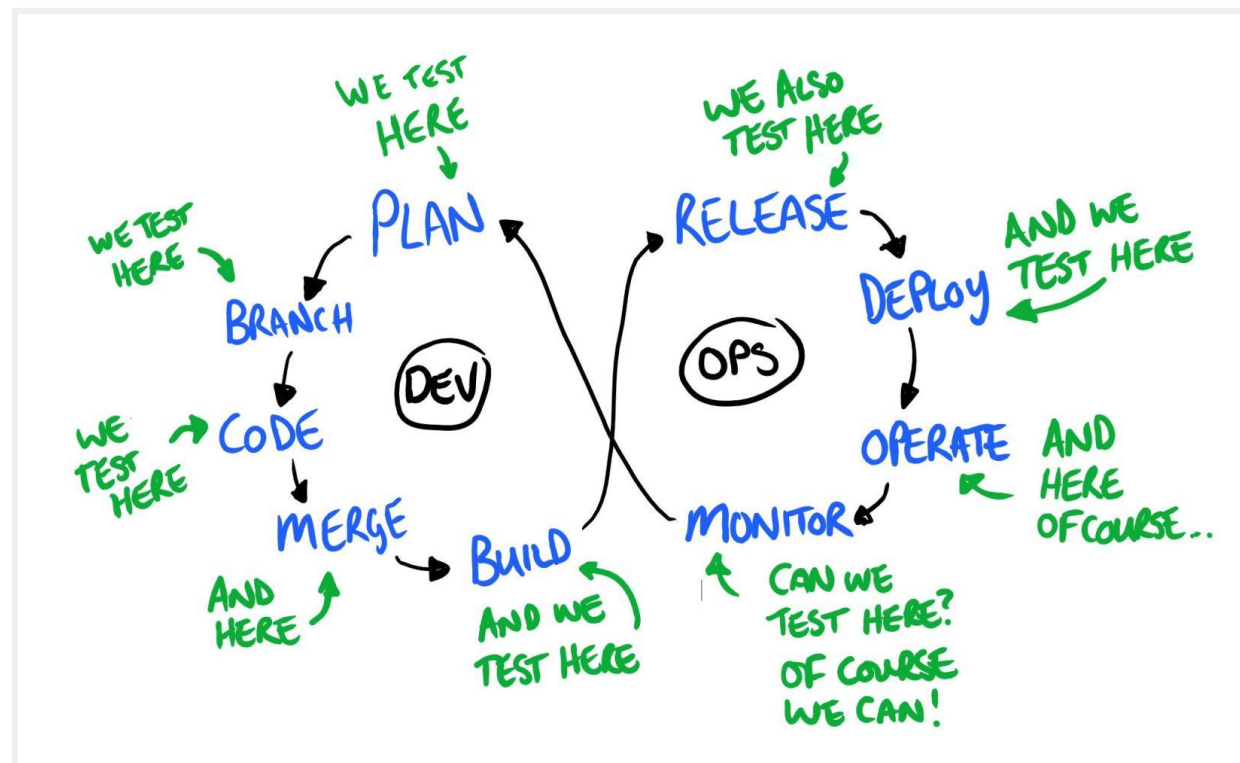
Definición

El **Agile Testing** son prácticas de Testing colaborativas que suceden continuamente, desde el inicio hasta la entrega y más allá, y que respaldan la entrega frecuente de valor a nuestros clientes. Las actividades de Testing se centran en crear calidad en el producto, usando rápidos ciclos de retroalimentación para validar nuestra comprensión. Las prácticas fortalecen y respaldan la idea de que todo el equipo es responsable de la calidad.



TESTING CONTINUO EN EL CICLO DEVOPS

El **Testing** es una parte integral del desarrollo de Software, entendemos las necesidades de nuestros clientes, las construimos, las entregamos y aprendemos como las usan y hacemos el **Testing parte integral del ciclo de DevOps**.



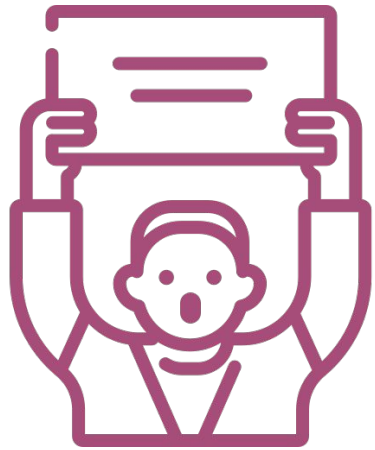
Fuente: Dan Ashby en su blog <https://danashby.co.uk/2016/10/19/continuous-testing-in-devops/>



LOS 10 PRINCIPIOS DEL AGILE TESTING

En el Agile Testing se introduce la idea de **10 principios** para los Agile Testers y no sólo para ellos sino **para cualquiera dentro del equipo**:

- Proporciona un feedback continuo.
- Entrega valor al cliente.
- Permite la comunicación cara a cara.
- Ten coraje.
- Sim.
- Practica la mejora continua.
- Responde al cambio.
- Auto-organízate.
- Céntrate en la gente.
- ¡Disfruta!



Testing Manifesto

Creado por Karen Greaves y Samantha Laing, el Testing Manifesto (el Manifiesto del Testing) refleja el cambio de mentalidad necesario para un enfoque de Agile Testing exitoso.



THE TESTING MANIFESTO

- **Valoramos el testing durante todo el proceso sobre las pruebas al final.**

Tradicionalmente el testing ha sido una fase que se ha producido al final del desarrollo. Sin embargo, **en la agilidad el testing es una actividad** que necesita darse junto con la codificación, la documentación y el resto de operaciones.

Si un tablero Kanban posee una columna de Testing, ten por seguro que el testing se está viendo como una fase y la actividad de los testers se está separando de la del resto del equipo. Como alternativa se recomienda una fase de Verificación donde una vez que el trabajo esté a DONE un compañero pueda revisarla.

- **Valoramos la prevención de errores en lugar de encontrar errores.**

Debe hacerse lo antes posible, normalmente la mayoría de los errores se dan por asunciones sin aclarar en la toma de requisitos.

- **Valoramos la comprensión de las pruebas sobre la verificación de la funcionalidad.**

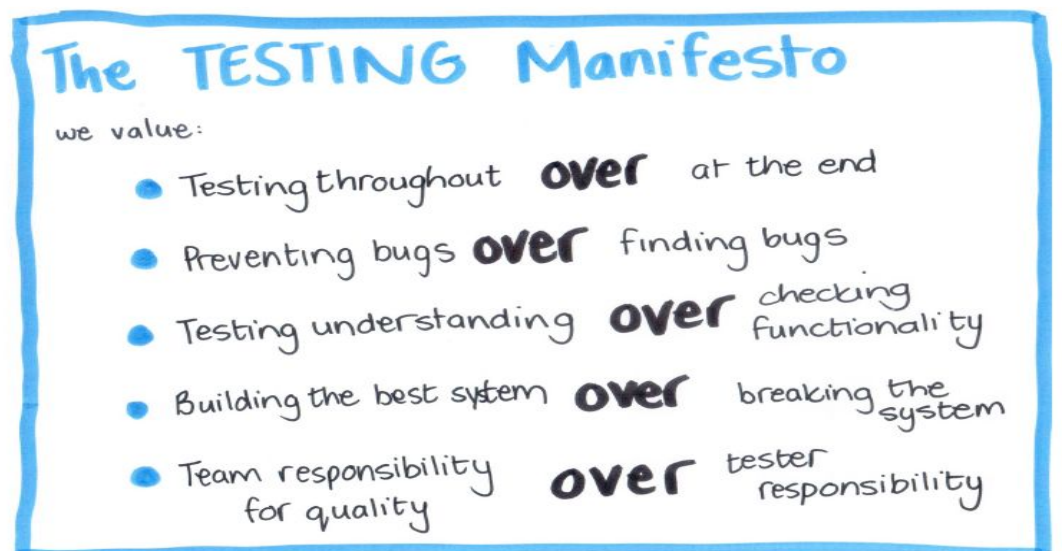
Los testers deben ser los representantes del cliente en cada decisión del diseño, asegurándose que la feature satisfaga las verdaderas necesidades de estos y no solo las especificaciones.

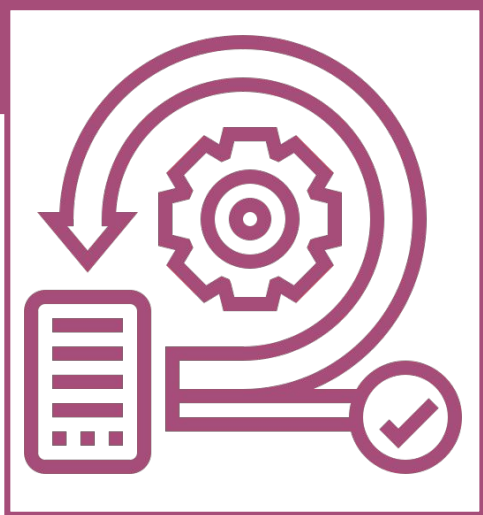
- **Valoramos construir el mejor sistema sobre romper el sistema.**

Tradicionalmente los Testers se han centrado en buscar la forma de romper el sistema, enfocándose en el Testing Negativo. Sin embargo, un tester se debe enfocar en el objetivo principal: crear un producto que aporte valor y cumpla su función como se espera.

- **Valoramos la responsabilidad del equipo por la calidad sobre la responsabilidad del probador.**

Los testers por sí solos no pueden mejorar la calidad. Su función es la de determinar el nivel de calidad e informar de ella a los interesados. Sólo puedes mejorar la calidad uniendo tus esfuerzos a los del equipo.





Planificación de Pruebas en Contextos Ágiles

Para planificar las pruebas de forma efectiva, un equipo necesita considerar su contexto. Para entender tu contexto piensa en tres aspectos de este:



EL EQUIPO

No todos los equipos que se crean son iguales. Podemos tener 2 casos:

1. **Equipo pequeños.**
 - a. Ubicados en la misma sala/edificio.
 - b. Buenas oportunidad para aprender los valores del resto y compartirlos.
 - c. La planificación es mucho más fácil.
2. **Equipos grandes y distribuidos.**
 - a. Trabajan en múltiples proyectos y con múltiples equipos.
 - b. Surge la necesidad de tener especialistas en el equipo.
 - c. Se incrementan las dependencias.
 - d. La planificación se vuelve más compleja.

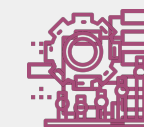


EL PRODUCTO

El nivel de calidad que las partes interesadas quieren depende del producto así como del tipo y cantidad de pruebas que pudiera ser requerido.

Consideraciones del producto:

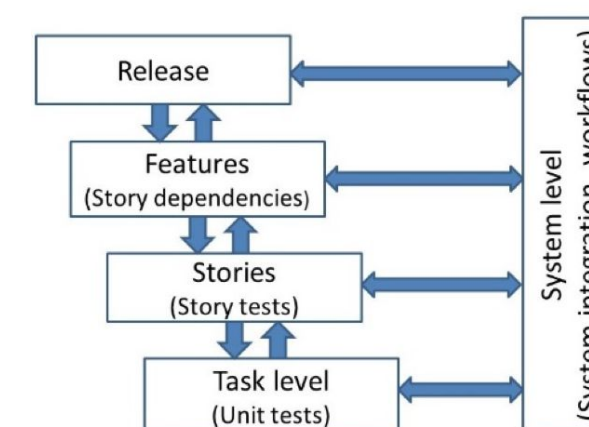
- **Tamaño:** ¿Cuánta gente lo va a usar? ¿Está integrado con Apps Externas?
- **Riesgos:** Para reducir riesgos los equipos necesitan conocer el dominio del negocio.
 - Los expertos de dominio deben ayudar en la planificación de las pruebas.
 - ¿Tiene idea el equipo de como se usa el producto?
 - ¿Tienen todos los miembros del equipo conocimiento del dominio?



NIVELES DE DETALLE

El **Testing diferentes niveles requiere de una planificación extra**, los niveles identificados son:

- **Nivel de Release/Feature**, las actividades deben ser coordinadas entre equipos.
- **Nivel de Historias**, emplear la Specification By Example para incrementar el entendimiento de la historia y convertirla en pruebas.
- **Nivel de Tareas**, usar TDD para escribir pruebas unitarias de bajo nivel. También se puede emplear DDD como alternativa.





Guiar el desarrollo con ejemplos

Ejemplos concretos del comportamiento del sistema de lo deseado y lo no deseado ayuda a construir un entendimiento compartido de cada feature e historia.



BDD

BDD (Behavior Driven Development)

captura ejemplos de escenarios en un lenguaje natural y específico del dominio.

Su sintaxis es:

GIVEN: Precondiciones

WHEN: Disparadores

THEN: Post Condiciones

Ejemplo:

GIVEN que he hecho una compra por valor de 9,75€.

WHEN entrego al cajero 10 €.

THEN espero 0,25€ de vuelta.



ATDD

ATDD (Acceptance Test-Driven Development)

es una forma más genérica de guiar el desarrollo con ejemplos sin un lenguaje o reglas estrictas. Se lleva a cabo:

- **ANTES** de implementar una necesidad todos los miembros del equipo contribuyen para generar escenarios de cómo se comportará dicha necesidad.
- **DESPUÉS** el equipo convierte esos escenarios en pruebas de Aceptación Automatizadas.



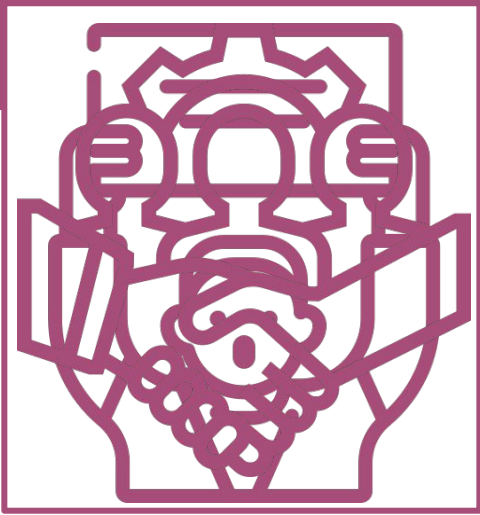
SBE

SBE (Specification By Example), se empieza identificando los objetivos entorno a la Historia usando un planteamiento como el Impact Mapping.

A partir de aquí **el equipo obtiene ejemplos clave que se convierten en especificaciones.**

A medida que avanza el desarrollo, los ejemplos se perfeccionan **y se convierten en especificaciones ejecutables** que permiten validar el producto con frecuencia.

Estos ejemplos ejecutables junto con BDD y ATDD, **se convierten en documentación viva de la aplicación.**



Habilitando la Colaboración

La colaboración dentro del equipo y entre los equipos es una de las piedras angulares para hacer que los equipos ágiles tengan éxito. Sin embargo, nos encontramos con que muchos equipos no saben como empezar a construir estas relaciones. Te presentamos algunas prácticas que te pueden ayudar en ello.



PRÁCTICAS

Dinámica de los 3 Amigos

Es una herramienta poderosa para compatir un entendimiento compartidos sobre features, historias y cómo encajan estas en el producto. La dinámica de los 3 Amigos **consiste en una reunión entre el tester, el programador y el experto de negocio para examinar el incremento del producto antes, durante y después del desarrollo.**

Otros roles que pueden participar son: UX, un experto de Big Data, experto en DevOps, etc.

Dinámica del Impact Mapping

Ayuda a decidir qué features debemos construir y quizás incluso determinar qué prioridad deben tener. Empezamos con el objetivo de una característica, el “**why**”, después identificamos quién podía ayudarnos a alcanzar ese objetivo “**who**”. Por cada “who” preguntamos cómo podrían ayudarnos en alcanzar este objetivo “**how**”. Para finalizar pensamos en qué entregables podrían resulta de cada impacto “**what**”.

Dinámica del Example Mapping

Es una estupenda forma de explorar una feature así como el valor que debería entregar. Mediante esta dinámica se producen conversaciones que concretan ejemplos que se usan para clarificar nuestro entendimiento de las reglas de negocio.

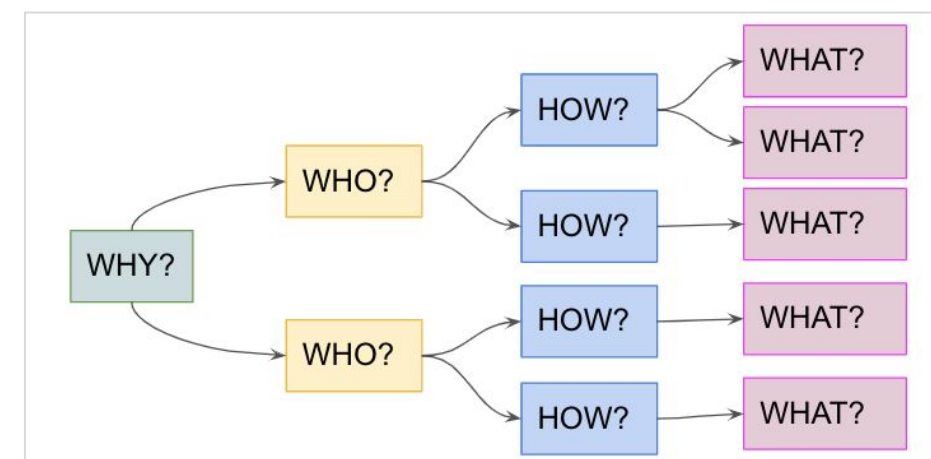


Figura 1: Impact Mapping

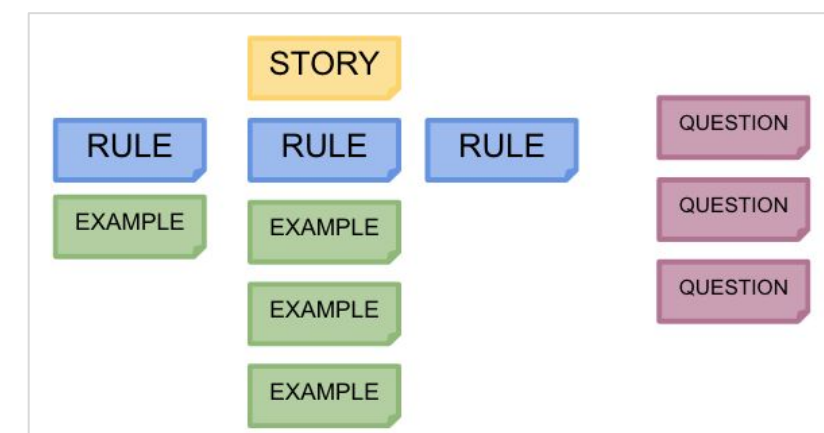


Figura 2: Example Mapping

Pruebas Exploratorias

En las pruebas exploratorias, **una persona interactúa con el sistema y observa el comportamiento real, diseñando pequeños experimentos**. Según lo que aprenden, adaptan el experimento y continúan aprendiendo más sobre el sistema. En el proceso, pueden hacer descubrimientos sorprendentes, incluyendo implicaciones de interacciones que nadie había considerado. **Las pruebas exploratorias exponen malentendidos sobre lo que se supone que debe hacer el software.**



TÉCNICAS

Personas, Jobs y Roles

Asumir el rol de una persona, **permite al equipo probar un producto desde la piel de otra persona y ver las cosas desde una perspectiva diferente**, pudiéndose superar la barrera a diferentes sesgos cognitivos inconscientes tales como el sesgo de confirmación.

Una **persona es un usuario ficticio que el equipo crea** y al que otorga ciertas características como una edad, nivel cultural, educación, experiencia, diferentes rasgos de personalidad, etc. Principalmente basados en el cliente objetivo al que va orientado el producto.

Combinar la persona con oficios o roles da un resultado mucho más favorable en las pruebas exploratorias.



Explorar en Parejas o Grupos

Cuantos más ojos haya sobre el problema mayores oportunidad de que se encuentre algo habrá.

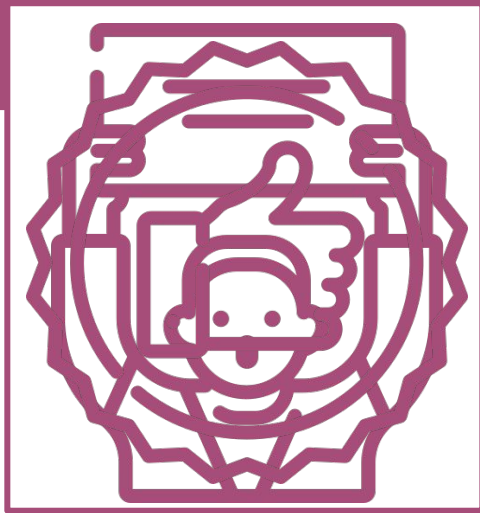
Charters

Los charters te ayudan a organizar la información que necesitas para aprender sobre tu aplicación adecuadamente en sesiones de tiempo fijo. Funcionan mejor cuando se utiliza junto con la técnica Personas. Siguen el esquema:

EXPLORE el carrito de la compra de la web

WITH exploits de scripting

TO DISCOVER cualquier vulnerabilidad



Atributos de Calidad del Testing

Un atributo de calidad, también conocido como requisito no funcional, **describe las propiedades bajo las cuales una característica debe operar**. Podemos pensar en él como una restricción que el equipo debe considerar con cada Feature o Historia.



TIPOS

1. **DESARROLLO**, estos atributos vienen a representar el **CÓMO** desarrollamos nuestro código e incluyen:

- a. Mantenibilidad del Código.
- b. Reusabilidad del Código.
- c. Testeabilidad del Código.

2. **OPERACIONALES**, son atributos que encaran la tecnología e incluyen:

Disponibilidad	Interoperabilidad	Escalabilidad	Rendimiento
Seguridad	Confiabilidad	Robustez	Recuperabilidad
Instalabilidad	Usabilidad	Protectibilidad	

Los equipos que esperan demasiado en el ciclo para prob

ar estos atributos (normalmente al final antes de lanzar el producto) **se topan con un bloqueo total**. Este tipo de problemas son normalmente problemas de diseño y no pueden ser reparados tan tarde.

Para mitigar los riesgos el equipo debe colaborar de forma temprana, siendo proactivo. No esperes a que el Product Owner inicie la conversación.



Visualización de una Estrategia de Automatización de Pruebas

La Automatización de Pruebas es un reto constante para los equipos de desarrollo de software en cualquier parte. **Te mostramos cómo el uso de Modelos Visuales pueden ayudar a tu equipo a diseñar su propia estrategia de automatización.**



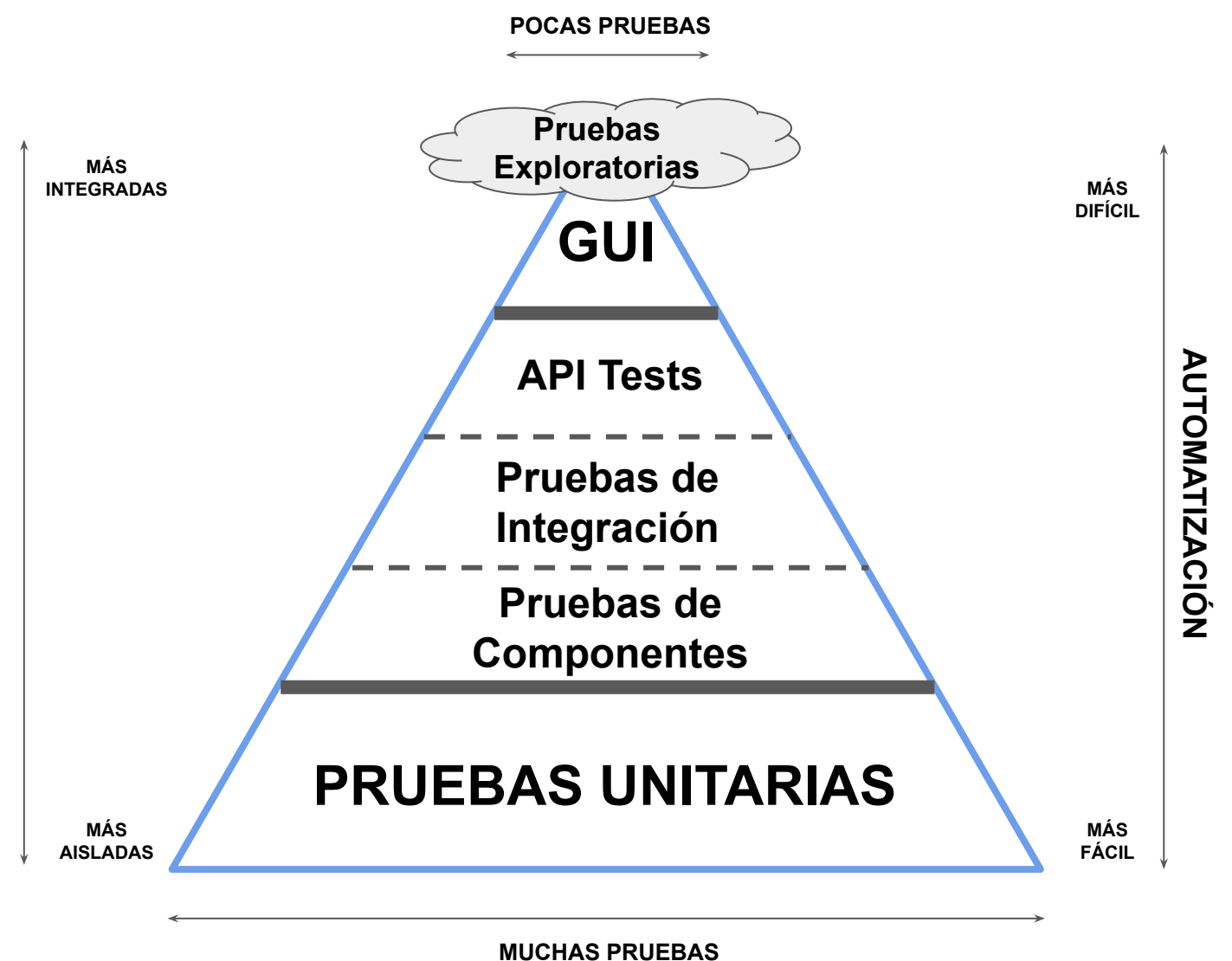
PIRÁMIDE DE AUTOMATIZACIÓN DE PRUEBAS DE MIKE COHN

Según la Pirámide de Mike Cohn **el grueso del testing** de cualquier proyecto **deberían ser las Pruebas Unitarias**, luego las **Pruebas de Componentes, Integración y API**. Y donde se debería poner menos esfuerzo será en las de **Interfaz de Usuario (GUI)**. **¿Por qué?**

Se trata de **anticiparse a los errores** que se puedan producir y tener controlados los elementos al más bajo nivel ya que esto nos asegura que no se vayan a propagar hacia arriba.

Además, la amplia cobertura de Pruebas Unitarias **nos documenta nuestro producto**. Hace que sepamos qué está fallando, dónde está el fallo y facilita su lectura (*incluso para las nuevas incorporaciones en el equipo*).

Normalmente **podemos encontrar un planteamiento diferente a la pirámide de Mike Cohn** y este no tiene por qué estar mal necesariamente (*pirámide invertida o reloj de arena*), aunque si la automatización actual no cubre las necesidades del cliente la imagen visual de nuestra estrategia de automatización de pruebas puede ayudarnos a detectar qué cambios son necesarios.





El Agile Tester

A diferencia del tester tradicional encargado de elaborar suites de prueba (a partir de diseños funcionales), y luego ser un mero ejecutor de checklist. El Agile Tester trabaja con el equipo siendo su prioridad principal la calidad y domina las herramientas de automatización, así como las metodologías ágiles.



CARACTERÍSTICAS Y HABILIDADES DEL AGILE TESTER

Las **principales características** de un Agile Tester son:

- Trabaja codo con codo con los Stakeholders.
- Proporciona experiencia en las actividades de testing.
- Se empareja con otros desarrolladores y testers.
- Dan apoyo al Product Owner.
- Organiza y adapta la estrategia de QA general.
- Obtiene buenos datos de las pruebas.
- Identifica riesgos.
- Es experto en guiar el desarrollo con ejemplos empleando para ello TDD, ATDD y SBE.
- Trabajo con el equipo para asegurar la cabay observabilidad de las pruebas para poder probarlas y monitorizarlas en Producción.
- Tiene pasión por la calidad y las pruebas, abogan por ellas y las defienden.



- **SHAPED SKILLS**

El Agile Tester tiene una amplia base de habilidades y conocimientos generales que apoyan el conocimiento profundo en el área del Testing.

Las **principales habilidades** de un Agile Testers son:

- **Colaboración**, para participar en prácticas como Mind Mapping o Example Mapping.
- **Facilitación**, para ayudar a los miembros del equipo a comunicarse mejor.
- **Enseñanza**, para compartir su conocimiento con otros miembros del equipo.
- **Coaching**, para ayudar al equipo a identificar problemas y diseñar experimentos para mejorar.
- **Comunicación**, para dar y recibir feedback de forma efectiva.